

LOONGREFLECT: Boosting Long-Horizon Reflection in Search Agents via Global Perspective Distillation

Zhixin Zhang^{*1,2,3}, Xinke Jiang^{*1,2,3}, Zhibang Yang^{*1,2}, Weixuan Xu^{1,2,3},
Guohong Qiu^{1,2,3}, Xu Chu^{†1,3,4}, Junfeng Zhao^{†1,3}, Yasha Wang^{†2,5}

¹School of Computer Science, Peking University, Beijing, China

²National Engineering Research Center for Software Engineering, Peking University, Beijing, China

³Key Laboratory of High Confidence Software Technologies, Ministry of Education, Beijing, China

⁴Center on Frontiers of Computing Studies, Peking University, Beijing, China

⁵Peking University Information Technology Institute (Tianjin Binhai), Tianjin, China

{zhixinzhang25,xinkejiang,yangzb}@stu.pku.edu.cn,

{chu_xu, zhaojf, wangyasha}@pku.edu.cn

Abstract

Large language model agents increasingly rely on long-horizon reasoning to solve complex tasks involving planning, tool use, and memory. A critical capability in such settings is **reflection**: assessing trajectory progress, identifying missing evidence and unreliable intermediate states, and deciding whether to continue, revise, or abandon the current branch. Learning effective reflection, however, is challenging because reflection is performed locally within the current branch, whereas its utility can only be determined by its contribution to the final trajectory outcome. This local-global mismatch makes outcome-based reinforcement learning provide only local, sparse and delayed supervision for reflective decisions. To solve these, we propose **LOONGREFLECT**, a training framework that formulates reflection as a *memory-control policy*. The agent operates over a reversible trajectory tree using explicit **<reflect>** and **<backtrack>** actions. Reflection consolidates verified facts, missing evidence, and branch-specific risks into working memory, while backtracking removes an unreliable branch from the active context and preserves a concise corrective lesson. To learn this policy, LOONGREFLECT combines two complementary signals through a look-ahead, extragradient-style coordination mechanism. A *fast channel* distills globally informed reflective behavior from a privileged teacher, with supervision restricted to reflection and backtracking tokens. A *slow channel* optimizes complete trajectories using outcome-based GRPO, aligning local control decisions with final task success. Experiments on multi-hop retrieval-augmented generation and mathematical reasoning benchmarks demonstrate consistent improvements over outcome-only reinforcement learning and self-distillation baselines.

1 Introduction

Large language models (LLMs) have emerged as the core policy components of agents that reason, retrieve information, and maintain intermediate memory (Yao et al. 2022; Schick et al. 2023; Asai et al. 2024; Jiang et al. 2025b). As these systems are increasingly deployed on open-ended,

^{*}These authors contributed equally.

[†]Corresponding authors.

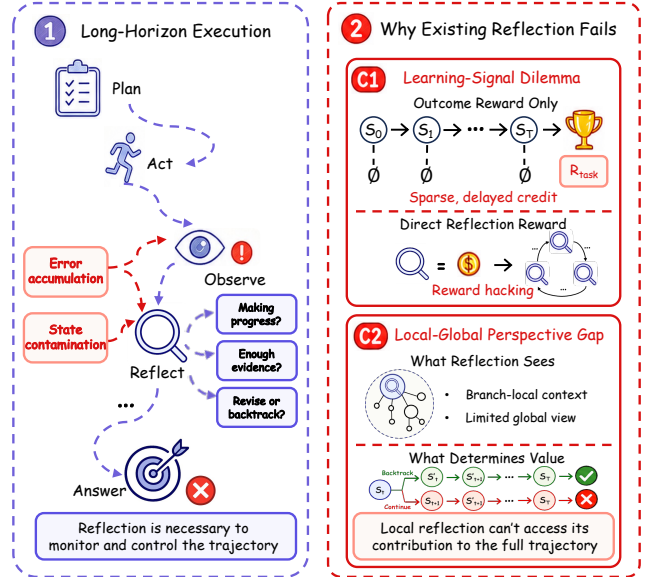


Figure 1: Reflective decisions are made from branch-local context, whereas their value is determined by their contribution to the final trajectory outcome. This local-global mismatch leaves outcome-based learning with local, sparse and delayed supervision.

multi-step tasks, long-horizon reasoning becomes critical. In such settings, an agent must do more than generate the next action: it must also continually assess whether its current trajectory is making progress, whether the available evidence is sufficient, and whether an earlier branch should be revised or abandoned, which is referred to as **reflection**.

Reflection is not merely additional reasoning text; it is a control process over the trajectory itself. An effective reflection step should consolidate verified facts, identify missing evidence, diagnose branch-specific risks, and decide whether the current branch should be continued, revised, or abandoned (Jiang et al. 2025a,b; Zhang et al. 2026). This capa-

bility is particularly important over long horizons, where localized errors—such as irrelevant retrievals, spurious entity associations, or stale memory updates—can enter the active context and influence many subsequent decisions. Without an explicit mechanism for isolating and correcting such errors, state contamination compounds as the trajectory grows, becoming a major obstacle to reliable agent behavior (Zhu et al. 2025; Jiang et al. 2025a,b; Zhang et al. 2026).

Despite its importance, learning effective reflection faces two fundamental challenges, as illustrated in Figure 1. ❶ **(C1) Learning-signal dilemma.** The value of a reflective decision is mediated by many subsequent actions and is revealed only through the final task outcome. Outcome-based reinforcement learning therefore provides sparse, delayed, and weakly attributable supervision for reflection. Yet assigning reflection an explicit intermediate reward is equally problematic and prone to reward hacking, as it may encourage excessive or superficial reflection without improving task success. ❷ **(C2) Local-global perspective gap.** Reflection is performed from the context of the current branch, whereas its true value depends on how that branch contributes to the complete trajectory. A local reflector cannot directly observe whether continuing, revising, or abandoning the branch will ultimately improve the outcome. Together, these challenges create a fundamental mismatch: **effective reflection must be learned from dense local guidance, yet evaluated and calibrated from a global trajectory perspective.**

Existing approaches address only part of this problem. ❶ One line of work improves intermediate judgment through step-level verification or action evaluation (Lightman et al. 2024; Wang et al. 2024; Liu et al. 2026b,a; Guo et al. 2025). ❷ Another provides denser learning signals through privileged feedback or error-localized supervision (Agarwal et al. 2024; Zhao et al. 2026a,b; Li, Hong, and Lan 2026; Huang et al. 2026). However, these approaches either treat reflection primarily as **local judgment** or rely on **misaligned supervision signals** that are not fully consistent with long-horizon outcomes. They therefore fall short of learning reflection as trajectory-level control for long-horizon agents.

To address **C1&C2**, we propose **LOONGREFLECT**, a training framework that formulates reflection as a **memory-control policy**, designed for long-horizon setting. Rather than treating reflection as unconstrained verbal feedback, LOONGREFLECT turns it into explicit, structured decisions over what the agent should **retain, revise, or discard from its reasoning memory**. Specifically, the agent operates over a **reversible trajectory tree** with an explicit active path and two control actions: **<reflect>** and **<backtrack>**. The **<reflect>** action consolidates verified facts, missing evidence, and branch-specific risks into working memory. When the current branch is deemed unreliable, the **<backtrack>** action removes its contaminated suffix from the active context, restores a validated state, and preserves a concise corrective lesson for subsequent decisions.

We train this memory-control policy through two complementary learning channels designed to resolve the two challenges jointly. ❶ A **fast channel** distills reflective behavior from a privileged teacher, with supervision restricted to reflection and backtracking tokens. This token-level su-

pervision provides the **dense** learning signal missing from outcome-based reinforcement learning, thereby addressing **C1**. Moreover, because the teacher observes the trajectory tree and terminal outcome, it can evaluate a local branch from a **global** perspective, thereby addressing **C2**. To prevent answer imitation, the teacher produces answer-masked feedback that focuses exclusively on state diagnosis, missing evidence, branch risks, and control decisions. ❷ A **slow channel** applies outcome-based GRPO to complete trajectories, ensuring that the distilled reflective behavior remains aligned with final task success. We coordinate the two channels through a look-ahead, extragradient-style update, in which the slow objective evaluates and calibrates the update direction proposed by the fast channel before the combined update is committed. Our contributions are as follows:

- We formulate reflection as a **memory-control policy** for long-horizon agents and instantiate it through explicit **<reflect>** and **<backtrack>** actions over a reversible trajectory tree.
- We introduce a two-channel learning framework that combines answer-masked teacher distillation for dense, globally informed reflective supervision with outcome-based GRPO for trajectory-level alignment, coordinated through a look-ahead, extragradient-style update.
- LOONGREFLECT consistently outperforms outcome-only RL and self-distillation baselines on multi-hop RAG and mathematical benchmarks, demonstrating the contributions of structured reflection, reversible backtracking, and two-channel optimization.

2 Related Work

Long-Horizon Agents

Large language models increasingly serve as the policy backbone of agents that interleave reasoning, retrieval, and tool use over extended interactions (Yao et al. 2022; Schick et al. 2023; Asai et al. 2024). Recent studies apply reinforcement learning to optimize long-horizon interaction and search policies directly from task outcomes (Chen et al. 2025; Gao et al. 2025). Memory-oriented approaches further support extended execution by compressing interaction histories or explicitly manipulating intermediate states (Jiang et al. 2025a; Zhang et al. 2026). For example, MEM1 jointly optimizes reasoning and compact memory, while AgenticRAG-R1 introduces stack-based memory with planning, summarization, and backtracking actions (Zhou et al. 2025; Jiang et al. 2025b). However, these methods largely rely on task-level objectives or generic action-quality signals, offering limited supervision for identifying which local decision corrupted the execution state or whether a recovery action was effective. A complementary line of work provides denser corrective signals through on-policy distillation: OPSD distills a privileged self-teacher on trajectories generated by the current policy, whereas ROSD localizes supervision to the first erroneous span to preserve valid prefixes (Zhao et al. 2026a,b). Nevertheless, these methods are primarily designed for linear reasoning traces and **provide limited support for reversible branch-level recovery or for aligning local corrections with delayed task outcomes.**

Self-Reflection and Self-Correction

Self-reflection and self-correction have been widely studied as inference-time mechanisms for iterative diagnosis and revision. Self-Refine repeatedly critiques and improves an initial response, while Reflexion stores verbal feedback from previous attempts as episodic experience (Madaan et al. 2023; Shinn et al. 2023). External tools can further ground reflective feedback and improve correction reliability (Gou et al. 2024). Nevertheless, intrinsic self-correction remains unreliable when models must evaluate their own generations without verifiable feedback (Kamoi et al. 2024). Recent learning-based methods therefore introduce explicit supervision for reflection and intermediate decisions. RISE jointly trains solution generation and self-verification on current-policy outputs (Liu et al. 2026b), while Agentic Critical Training learns to distinguish superior actions from suboptimal alternatives (Liu et al. 2026a). Process-supervised approaches further estimate intermediate action quality through step-level evaluation or automatically constructed reward models (Xiong et al. 2024; Deng et al. 2025). Although these methods provide finer-grained supervision than terminal rewards, they mainly **target verification or action selection, with limited support for explicit state recovery and aligning local reflection with delayed long-horizon outcomes.**

3 Method

In this section, we present LOONGREFLECT, as illustrated in Figure 2. It consists of two components: ❶ a memory-control framework that supports structured reflection through a reversible trajectory tree and explicit reflection actions; and ❷ a two-channel optimization framework that combines dense local distillation with outcome-based global correction through look-ahead updates.

Problem Formulation

Given a task x , LOONGREFLECT models a long-horizon agent as both an *execution policy* and a *memory-control policy*. In addition to reasoning, retrieval, and answer generation, the agent must determine which information accumulated during execution should be retained, revised, or discarded. We therefore equip the agent with a reversible trajectory state that can be explicitly modified through reflection and backtracking. At step t , the agent state is defined as

$$\mathbf{z}_t = (x, \mathcal{T}_t, P_t, \mathbf{m}_t), \quad (1)$$

where $\mathcal{T}_t$ denotes the accumulated trajectory tree, P_t the active execution path, and $\mathbf{m}_t$ the working memory compressed from that path. Although the agent executes actions sequentially along P_t , its complete interaction history is not necessarily linear. When the agent backtracks to an earlier state and resumes execution, the newly generated suffix forms an alternative branch, while the abandoned suffix remains in $\mathcal{T}_t$ as an inactive branch. Repeated recovery therefore naturally induces a tree structure, with each root-to-node path representing a distinct execution branch.

Only the active path P_t and its compressed memory $\mathbf{m}_t$ are exposed to subsequent generation. Inactive branches are excluded from the active context to prevent unreliable states

from influencing future decisions, but remain in $\mathcal{T}_t$ for error diagnosis and reflective supervision. This separation enables the agent to recover from a corrupted branch without discarding the information needed to learn from it.

Conditioned on $\mathbf{z}_t$, the agent selects actions from both execution and control spaces:

$$a_t \sim \pi_\theta(\cdot | \mathbf{z}_t), \quad a_t \in \mathcal{A}_{\text{exec}} \cup \mathcal{A}_{\text{ctrl}}, \quad (2)$$

where $\mathcal{A}_{\text{exec}}$ contains reasoning, tool-use, and answer actions, while $\mathcal{A}_{\text{ctrl}}$ contains reflection and backtracking actions. Execution actions advance task completion, whereas control actions assess and regulate the validity of the active state. The overall learning objective is to maximize the terminal task reward:

$$\max_{\theta} \mathbb{E}_{\tau \sim \pi_\theta(\cdot | x)} [R(\tau)], \quad (3)$$

where $R(\tau)$ is determined by the final outcome of trajectory τ . This objective alone, however, provides only sparse trajectory-level feedback for reflection decisions made at intermediate states. We therefore introduce a structured memory-control mechanism together with a two-channel optimization procedure, as described in the following sections.

Reflection as Memory Control

To support reflection over intermediate states, LOONGREFLECT represents agent memory as a reversible trajectory tree. At step t , the execution state consists of a trajectory tree $\mathcal{T}_t = (\mathcal{V}_t, \mathcal{E}_t)$, an active path P_t , compressed memory $\mathbf{m}_t$, and archived branches $\mathcal{B}_t$. The LLM context is serialized as

$$c_t = \text{Serialize}(x, P_t, \mathbf{m}_t). \quad (4)$$

Thus, the model accesses only the task, active path, and compressed memory, while $\mathcal{T}_t$ and $\mathcal{B}_t$ remain internal states for future diagnosis and recovery. This decouples current decision making from historical branches. Beyond task-solving actions, the agent may trigger reflection controls:

$$a_t^{\text{ctrl}} \in \mathcal{A}_{\text{ctrl}} = \{ \langle \text{reflect} \rangle, \langle \text{backtrack} \rangle \}. \quad (5)$$

These actions form the memory-control mechanism:

- **<reflect>** diagnoses the active state by summarizing evidence, identifying missing information or faulty assumptions, and proposing the next control decision.
- **<backtrack>** executes recovery when reflection determines that the current state is unreliable, rolling the active trajectory back to a trustworthy prefix and removing

Concretely, **<reflect>** produces a structured summary.

$$\mathbf{r}_t = (e_t^{\text{ver}}, q_t^{\text{risk}}, j_t^{\text{ret}}, d_t^{\text{ctrl}}), \quad (6)$$

$$d_t^{\text{ctrl}} \in \{ \text{continue}, \text{backtrack} \}.$$

where e_t^{ver} , q_t^{risk} , j_t^{ret} , and d_t^{ctrl} denote evidence, risk, return point, and control intent, respectively. The control intent determines whether to continue along the active path or invoke **<backtrack>** for recovery, making reflection a state-control diagnosis rather than a generic critique. When

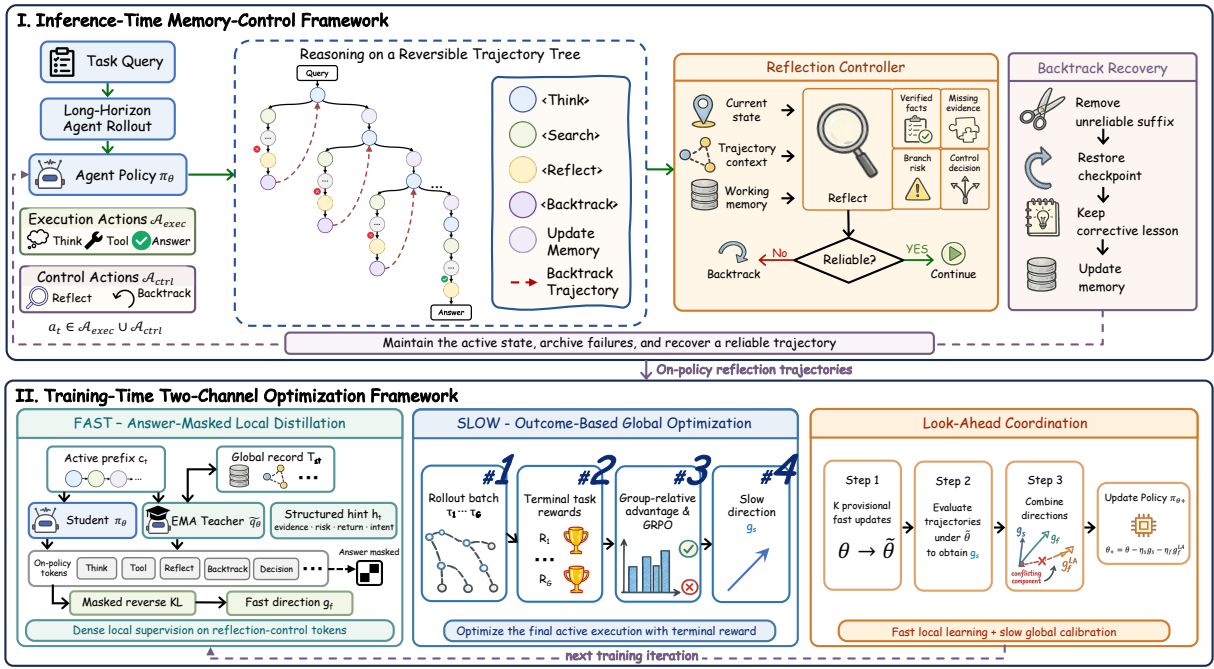


Figure 2: Overview of the LOONGREFLECT framework.

reliable, the active path grows forward; otherwise, the system restores a reliable prefix and performs the transition

$$P_{t+1} = P_j \oplus u_{j:t}, \quad \mathcal{B}_{t+1} = \mathcal{B}_t \cup \{P_{j+1:t}\}, \quad (7)$$

where P_j is the recovered active prefix, $P_{j+1:t}$ is the invalidated suffix removed from the active context, and $u_{j:t}$ is a compact corrective update distilled from the current reflection, such as the decisive contradiction, the falsified assumption, or the constraint that the next attempt must satisfy.

Two-Channel Optimization for Reflection Learning

Fast Local Supervision. To provide dense supervision for reflection learning, LOONGREFLECT introduces a fast channel over reflection control tokens. At step t , the student generates an active-state prefix with context $c_t = (x, P_t, \mathbf{m}_t)$. A teacher then constructs an answer-masked structured control label from the execution history:

$$\mathbf{h}_t = \mathcal{H}_\phi(x, \mathcal{T}_{\leq t}, P_t, \mathbf{m}_t), \quad (8)$$

where $\mathcal{T}_{\leq t}$ includes both the current active path and archived historical branches, and $\mathcal{H}_\phi$ denotes a privileged hint constructor implemented by either an auxiliary LLM or a rule-based feedback module. Its output shares the same schema as the reflect summary,

$$\mathbf{h}_t = (e_t^{\text{ver}}, q_t^{\text{risk}}, j_t^{\text{ret}}, d_t^{\text{ctrl}}), \quad (9)$$

but is constructed from privileged access to the global execution record while masking the final answer, so the fast channel supervises local diagnosis and recovery rather than answer generation. The student and teacher then evaluate the same continuation. Let $y_{t,k}$ denote the k -th generated token

at step t . We define:

$$\begin{aligned} \ell_{t,k} &= \log \pi_\theta(y_{t,k} | c_t, y_{t,<k}), \\ \bar{\ell}_{t,k} &= \log q_{\bar{\theta}}(y_{t,k} | c_t, \mathbf{h}_t, y_{t,<k}). \end{aligned} \quad (10)$$

where $q_{\bar{\theta}}$ is an EMA teacher of the policy. The teacher and student share the same on-policy prefix and continuation; the only additional information available to the teacher is the structured hint $\mathbf{h}_t$. Since the fast channel supervises only reflection-related spans, we define the token mask

$$m_{t,k}^{\text{ref}} = \mathbf{I}[y_{t,k} \in \text{Span}(\langle \text{reflect} \rangle, \langle \text{backtrack} \rangle)],$$

with normalization factor $Z = \sum_{t,k} m_{t,k}^{\text{ref}}$. Based on the teacher-student log-probability gap $\delta_{t,k} = \ell_{t,k} - \bar{\ell}_{t,k}$, we optimize a masked reverse-KL objective (a k3-style unbiased estimator) restricted to the reflection span:

$$\mathcal{L}_{\text{fast}} = \frac{1}{Z} \sum_{t,k} m_{t,k}^{\text{ref}} \min(\exp(-\delta_{t,k}) - 1 + \delta_{t,k}, c), \quad (11)$$

where the clipping constant c suppresses extreme token-ratio estimates. This objective is applied only to **<reflect>** and **<backtrack>** tokens, providing dense local supervision for state diagnosis and recovery decisions.

Slow Global Optimization. To calibrate the global utility of reflection, LOONGREFLECT introduces a slow channel that optimizes reflection decisions based on terminal trajectory outcomes. Unlike the fast channel, which supervises local state diagnosis, the slow channel evaluates whether these decisions improve final task success. Specifically, for each task x , we sample complete trajectories $\tau_1, \tau_2, \dots, \tau_G \sim \pi_\theta(\cdot |$

x) with terminal rewards $R_1, R_2, \dots, R_G$, where R_i is a task-level reward returned by the environment or an answer verifier. The slow channel thus evaluates reflection by trajectory-level success rather than the plausibility of individual control decisions. Given the trajectory group, we define the group-relative advantage as $\hat{A}_i = (R_i - \text{mean}_g(R_g)) / \text{std}_g(R_g)$.

This normalizes rewards within the sampled group and assigns credit based on relative trajectory quality. To ensure clear credit assignment, the slow objective is applied only to policy-generated tokens in the final active execution. Let ξ_i denote the policy token sequence of trajectory τ_i , including execution and reflection control tokens (**<reflect>** and **<backtrack>**), while excluding tool outputs, external observations, and controller updates. We then define the token ratio with respect to the old policy as

$$\rho_{i,k}(\theta) = \frac{\pi_\theta(y_{i,k} | c_{i,k})}{\pi_{\text{old}}(y_{i,k} | c_{i,k})}, \quad (12)$$

where $c_{i,k}$ denotes the conditional prefix for the k -th token in trajectory τ_i . Let $\bar{\rho}_{i,k}(\theta) = \text{clip}(\rho_{i,k}(\theta), 1 - \varepsilon, 1 + \varepsilon)$. The slow channel is optimized by

$$\mathcal{L}_{\text{slow}}(\theta) = -\frac{1}{G} \sum_{i=1}^G \frac{1}{|\xi_i|} \sum_{k \in \xi_i} \min\{\rho_{i,k}(\theta) \hat{A}_i, \bar{\rho}_{i,k}(\theta) \hat{A}_i\} + \beta D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}}).$$

Since $\hat{A}_i$ is derived from terminal outcomes, slow channel rewards reflection only when it improves overall performance.

Look-Ahead Coordination. The fast and slow channels provide local and global signals, but their update directions may conflict. LOONGREFLECT introduces look-ahead coordination, where slow channel calibrates fast update before optimization. Given θ , we obtain a provisional fast policy $\tilde{\theta} = \mathcal{U}_{\text{fast}}^K(\theta)$ by applying K inner fast-channel updates estimated direction:

$$g_f = (\theta - \tilde{\theta}) / \alpha, \quad (13)$$

where α is accumulated inner step size. We then evaluate trajectories under $\tilde{\theta}$ to derive slow-channel calibration direction

$$g_s = \nabla_{\tilde{\theta}} \mathcal{L}_{\text{slow}}(\tilde{\theta}). \quad (14)$$

Here, g_s identifies fast-policy updates that improve final outcomes, while g_f represents local reflection supervision. If g_f and g_s conflict, we remove the opposing component of g_f along g_s , yielding the calibrated fast direction

$$g_f^{\text{LA}} = \begin{cases} g_f - \frac{\langle g_f, g_s \rangle}{\|g_s\|_2^2} g_s, & \langle g_f, g_s \rangle < 0, \\ g_f, & \text{otherwise.} \end{cases} \quad (15)$$

This operation preserves fast updates aligned with the global objective while removing only conflicting components. Thus, look-ahead coordination calibrates rather than weakens the fast signal. After calibration, we return to the original parameters θ and apply the fused update

$$\theta^+ = \theta - \eta_s g_s - \eta_f g_f^{\text{LA}}, \quad (16)$$

where η_s and η_f denote the outer-step sizes for the slow and calibrated fast directions. The fast channel learns local reflection control, the slow channel provides global calibration, and look-ahead coordination aligns them before optimization.

4 Experiments

We evaluate LOONGREFLECT to determine whether globally supervised memory control improves long-horizon reflection to answer the four research questions:

- **RQ1:** Does LOONGREFLECT outperform SOTA baselines on both in and out-of-domain setting?
- **RQ2:** Does the learned reflection policy transfer beyond retrieval to mathematical reasoning?
- **RQ3:** How much do supervised fine-tuning and two-channel reinforcement learning each contribute?
- **RQ4:** Which memory-control and optimization components are responsible for the gains, and how sensitive is the method to its coordination hyperparameters?

Experimental Setup

1 Training and Evaluation Benchmarks. Following AgenticRAG-R1, we train on HotpotQA and 2WikiMultiHopQA after filtering questions that can be answered without retrieval or with a single trivial lookup (Yang et al. 2018; Ho et al. 2020; Jiang et al. 2025b). We evaluate on seven retrieval-augmented QA benchmarks. 2WikiMultiHopQA and HotpotQA are treated as in-domain, while Bamboogle, FRAMES, MuSiQue, Natural Questions (NQ), and TriviaQA measure transfer to compositional, open-domain, and distribution-shifted questions (Press et al. 2023; Krishna et al. 2025; Trivedi et al. 2022; Kwiatkowski et al. 2019; Joshi et al. 2017). We additionally use MATH and GSM8K to examine transfer to non-retrieval multi-step reasoning (Hendrycks et al. 2021; Cobbe et al. 2021).

2 SFT Data Construction. Before RL, we distill SFT trajectories from a locally deployed Qwen3-32B model. To elicit reflection during generation, whenever the teacher produces an incorrect answer before exhausting the maximum step budget, we replace answer action with **<reflect>** and let the rollout continue until it reaches the correct answer within the budget. We then apply two-stage rejection sampling to retain long, informative rollouts. First, we sample LOONGREFLECT trajectories and retain successful trajectories with at least one reflection and at least five interaction turns. Second, we resample the same questions with Search-R1 (Jin et al. 2025) and keep if it succeeds while Search-R1 fails.

3 Models and Baselines. We use Qwen2.5-3B and 7B instruction models (Yang et al. 2024); the ablations use Qwen2.5-3B. We compare methods from four paradigms: *no RAG* (Base and CoT), *naive RAG* (FS-RAG and FL-RAG), *agentic RAG* (ReAct, IRCot, TCRAG, and ReSearch), and *RL-based agentic RAG* (Search-R1, AEPO, ARPO, Mem1, and AgenticRAG-R1) (Wei et al. 2022; Khandelwal et al. 2019; Trivedi et al. 2023; Yao et al. 2022; Jiang et al. 2025a; Chen et al. 2026; Jin et al. 2025; Dong et al. 2025a,b; Zhou et al. 2025; Jiang et al. 2025b). For math reasoning, we further compare with RLSD (Yang et al. 2026).

4 Retrieval Setup and Metric. All retrieval methods use same English Wikipedia snapshot dated November 1, 2023, together with the same retriever, top- k setting, context budget, tool-call budget, decoding configuration, and answer normalizer. We report answer-level F1 (%)

Paradigm	Method	In-Domain F1 (%)		Out-of-Domain F1 (%)					Average F1 (%)
	Approach	2Wiki	HotpotQA	Bamboogle	FRAMES	MuSiQue	NQ	TriviaQA	Avg.
Qwen2.5-3B									
No RAG	Base CoT	23.98 18.90	24.08 23.82	9.45 20.80	8.01 7.16	9.70 10.47	14.27 16.53	38.84 39.62	18.33 19.61
Naive RAG	FS-RAG FL-RAG	15.47 16.80	25.85 26.78	10.48 11.05	10.42 9.19	7.64 7.29	19.84 21.93	45.38 48.50	19.30 20.22
Agentic RAG	ReAct	25.09	34.37	24.86	10.53	13.92	27.19	46.04	26.00
	IRCoT	15.89	24.50	25.27	6.79	12.43	27.86	49.19	23.13
	TCRAG	28.47	21.94	17.59	7.69	8.99	20.74	50.46	22.27
	ReSearch	27.23	33.96	15.09	10.00	9.47	34.61	53.93	26.33
RL-based	Search-R1	29.90	37.24	29.90	10.76	13.53	34.73	55.08	30.16
	AEPO	23.01	28.71	22.09	12.26	11.70	26.76	47.78	24.62
	ARPO	29.55	36.48	27.32	13.49	13.38	33.29	53.66	29.60
	Mem1	18.06	20.15	5.19	4.99	4.47	19.18	33.09	15.02
	AgenticRAG-R1	32.92	44.00	31.48	16.23	16.48	37.15	56.62	33.55
	LOONGREFLECT	48.01	56.17	35.25	23.51	31.02	55.37	73.72	46.15
Qwen2.5-7B									
No RAG	Base CoT	25.41 23.55	26.63 29.10	17.86 37.56	12.52 17.60	12.15 14.35	19.72 22.47	49.08 49.33	23.34 27.71
Naive RAG	FS-RAG FL-RAG	17.71 19.78	29.21 34.42	16.86 24.10	12.52 12.10	10.74 12.46	16.82 19.72	35.02 42.66	19.84 23.61
Agentic RAG	ReAct	27.51	42.81	27.63	15.29	19.34	30.01	54.55	31.02
	IRCoT	36.45	26.29	21.90	6.78	8.39	19.63	49.43	24.12
	TCRAG	29.70	40.83	25.13	16.46	17.56	29.01	54.78	30.50
	ReSearch	30.03	30.39	30.42	15.61	12.58	23.69	48.25	27.28
RL-based	Search-R1	35.03	38.89	42.04	18.01	19.08	29.59	55.91	34.08
	AEPO	19.88	13.85	13.24	7.24	5.85	9.93	17.53	12.50
	ARPO	30.71	25.20	32.94	12.18	12.71	17.80	40.16	24.53
	Mem1	25.29	29.98	36.50	14.15	14.13	26.38	51.04	28.21
	AgenticRAG-R1	38.34	45.15	49.21	19.44	22.01	23.60	58.45	36.60
	LOONGREFLECT	54.44	53.27	53.89	23.17	25.81	58.98	74.91	49.21

Table 1: Performance comparison (F1, %) on multi-hop and open-domain QA benchmarks across no-RAG, naive-RAG, agentic-RAG, and RL-based paradigms using Qwen2.5-3B and 7B. 2Wiki and HotpotQA are in-domain; Others are out-of-domain.

Main Result Analysis

To answer **RQ1**, Table 1 compares LOONGREFLECT with representative baselines across the seven QA benchmarks. LOONGREFLECT achieves the highest F1 on every benchmark with both model sizes. Its average F1 reaches 46.15 with Qwen2.5-3B and 49.21 with Qwen2.5-7B, exceeding the strongest baseline, AgenticRAG-R1, by 12.60 and 12.61 points, respectively. The improvement extends beyond the training distribution. On Qwen2.5-3B, LOONGREFLECT raises the in-domain average from 38.46 to 52.09 and the out-of-domain average from 31.59 to 43.77 relative to AgenticRAG-R1. On Qwen2.5-7B, the corresponding averages increase from 41.75 to 53.86 and from 34.54 to 47.35. The consistent gains across model scales and evaluation regimes indicate that explicit reflection generalize beyond the training benchmarks rather than merely fitting the in-domain tasks.

Transfer to Mathematical Reasoning

To answer **RQ2**, Table 3 evaluates whether the learned reflection policy transfers beyond retrieval. With Qwen2.5-3B, LOONGREFLECT obtains 56.0 F1 on MATH and 82.4 F1 on GSM8K. It improves over AgenticRAG-R1 by 1.2 and 1.8 points, and over RLSD by 2.4 and 1.7 points, respectively. Al-

though LOONGREFLECT is motivated by long-horizon search, the gains on both datasets suggest that its learned diagnosis and recovery behavior also benefits multi-step reasoning without external retrieval.

Contributions of the Training Stages

To answer **RQ3**, Table 4 separates the effects of curated supervised fine-tuning and two-channel reinforcement learning. SFT improves average F1 over the raw instruction model by 4.43 points for Qwen2.5-3B and 3.54 points for Qwen2.5-7B, establishing an initial policy for reflection and backtracking. Applying two-channel RL yields a further 11.39-point gain for 3B and a 7.94-point gain for 7B. Overall, the complete pipeline improves over the raw backbones by 15.82 and 11.48 points. These results show that SFT provides a useful reflective prior, while globally calibrated two-channel optimization contributes the larger performance gain.

Ablation and Analysis

❶ **Component Ablation.** To answer **RQ4**, Table 2 first examines the two memory-control actions. Removing reflection reduces average F1 from 46.15 to 30.84 (−15.31), while removing backtracking lowers it to 33.09 (−13.06). The former result shows the importance of diagnosing the active state;

Paradigm	Method Approach	In-Domain F1 (%)		Out-of-Domain F1 (%)					Average F1 (%) Avg.
		2Wiki	HotpotQA	Bamboogle	FRAMES	MusiQue	NQ	TriviaQA	
Full Model	LOONGREFLECT	48.01	56.17	35.25	23.51	31.02	55.37	73.72	46.15
Memory-Control Actions	w/o <reflect>	26.41	37.74	18.35	11.46	18.18	45.28	58.45	30.84
	w/o <backtrack>	33.67	45.89	19.22	15.18	14.78	49.22	53.67	33.09
Two-Channel Optimization	w/o Fast Reflection Distillation	37.50	47.13	32.58	25.00	13.64	63.33	64.37	40.51
	w/o Slow Outcome Optimization	30.12	55.29	25.29	21.43	20.73	54.65	66.28	39.11
	w/o Look-Ahead Coordination	36.26	49.84	31.93	19.08	27.43	55.94	68.01	41.21

Table 2: Component ablation on the seven QA benchmarks using Qwen2.5-3B (F1, %). The first group removes the two memory-control actions; the second removes one part of the two-channel optimization at a time.

Method	MATH	GSM8K
AgenticRAG-R1	54.8	80.6
RLSD (Yang et al. 2026)	53.6	80.7
LOONGREFLECT	56.0	82.4

Table 3: Transfer performance (F1,%) on MATH and GSM8K using Qwen2.5-3B.

Stage	Qwen2.5-3B	Qwen2.5-7B
RAW	30.33	37.73
SFT	34.76	41.27
SFT+RL	46.15	49.21

Table 4: Effect of successive training stages on average F1 (%) across the seven QA benchmarks.

the latter confirms that diagnosis alone is insufficient without an explicit mechanism for discarding an unreliable suffix and resuming from a validated state.

The optimization ablations are also consistently worse than the complete method. Removing slow outcome optimization, fast reflection distillation, or look-ahead coordination decreases average F1 by 7.04, 5.64, and 4.94 points, respectively. Thus, dense local teacher guidance and trajectory-level outcome optimization provide complementary supervision, while look-ahead coordination is necessary to suppress locally preferred updates that conflict with final task success.

② Optimization Dynamics. Figure 3 tracks the two learning signals over 100 training steps. Task reward rises over the course of training, while the distillation reward moves upward from a strongly negative initial value toward zero. Their concurrent improvement indicates that the policy increasingly follows the teacher’s local reflection guidance without sacrificing complete-trajectory outcomes, consistent with the intended division of labor between the fast and slow channels.

③ Hyperparameter Sensitivity. Figure 4 studies the number of inner fast updates K and the relative fast-direction weight $w = \eta_f/\eta_s$ in the fused outer update. With $w=1$, F1 rises from 41.26 at $K=1$ to 46.15 at $K=3$, before declining to 44.71 at $K=4$. With $K=3$, $w=1$ outperforms both $w=0.5$ and $w=2.0$. Too few inner updates provide insufficient local adaptation, whereas an excessive relative fast weight can dominate the outcome-aligned slow direction. We therefore use $K=3$ and $w=1$ in the main experiments.

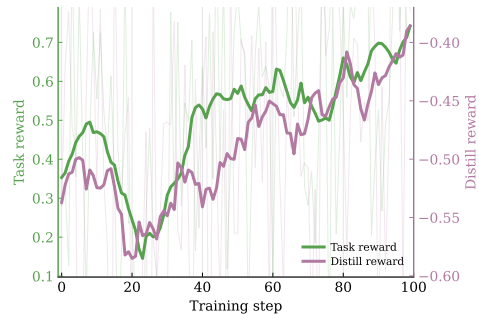


Figure 3: Two-channel optimization dynamics over 100 training steps using Qwen2.5-3B.

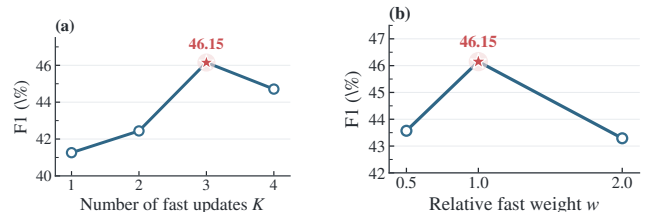


Figure 4: Sensitivity of performance to look-ahead coordination hyperparameters using 3B. K is the number of inner fast updates, and $w = \eta_f/\eta_s$ is computed as the ratio of the outer-step sizes for the calibrated fast and slow directions.

5 Conclusion

Long-horizon reflection faces the two challenges identified in the introduction: reflective decisions receive sparse and delayed outcome supervision, yet must be made from a branch-local context whose value depends on the complete trajectory. We introduced LOONGREFLECT to address this learning-signal dilemma and local-global perspective gap by treating reflection as an explicit memory-control policy. A reversible trajectory tree, together with **<reflect>** and **<backtrack>** actions, enables the agent to diagnose its active state, remove unreliable branches, and retain compact corrective information. For learning, the fast channel supplies dense, answer-masked supervision from a globally informed teacher, while the slow channel uses outcome-based GRPO to align local control with final task success. Look-ahead coordination reconciles these signals before the update is committed. Across seven retrieval-augmented QA bench-

marks, LOONGREFLECT consistently improves both Qwen2.5-3B and Qwen2.5-7B, including on out-of-domain tasks; with Qwen2.5-3B, it also transfers to two mathematical reasoning benchmarks. Training-stage and component ablations further verify the complementary contributions of structured reflection, reversible backtracking, local distillation, global outcome optimization, and their coordination.

References

- Agarwal, R.; Vieillard, N.; Zhou, Y.; Stanczyk, P.; Ramos Garea, S.; Geist, M.; and Bachem, O. 2024. On-policy distillation of language models: Learning from self-generated mistakes. In *International Conference on Learning Representations*, volume 2024, 21246–21263.
- Asai, A.; Wu, Z.; Wang, Y.; Sil, A.; and Hajishirzi, H. 2024. Self-rag: Learning to retrieve, generate, and critique through self-reflection. In *International conference on learning representations*, volume 2024, 9112–9141.
- Chen, K.; Cusumano-Towner, M.; Huval, B.; Petrenko, A.; Hamburger, J.; Koltun, V.; and Krähenbühl, P. 2025. Reinforcement learning for long-horizon interactive llm agents. *arXiv preprint arXiv:2502.01600*.
- Chen, M.; Sun, L.; Li, T.; Sun, H.; Zhu, C.; Wang, H.; Pan, J.; Zhang, W.; Chen, H.; Yang, F.; et al. 2026. Learning to reason with search for llms via reinforcement learning. *Advances in Neural Information Processing Systems*, 38: 85287–85307.
- Cobbe, K.; Kosaraju, V.; Bavarian, M.; Chen, M.; Jun, H.; Kaiser, L.; Plappert, M.; Tworek, J.; Hilton, J.; Nakano, R.; et al. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.
- Deng, Y.; Fan, S.; Wang, N.; Zhao, X.; and Ng, S. K. 2025. AgentPro: Enhancing LLM Agents with Automated Process Supervision. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 9992–10017.
- Dong, G.; Bao, L.; Wang, Z.; Zhao, K.; Li, X.; Jin, J.; Yang, J.; Mao, H.; Zhang, F.; Gai, K.; et al. 2025a. Agentic entropy-balanced policy optimization. *arXiv preprint arXiv:2510.14545*.
- Dong, G.; Mao, H.; Ma, K.; Bao, L.; Chen, Y.; Wang, Z.; Chen, Z.; Du, J.; Wang, H.; Zhang, F.; et al. 2025b. Agentic reinforced policy optimization. *arXiv preprint arXiv:2507.19849*.
- Gao, J.; Fu, W.; Xie, M.; Xu, S.; He, C.; Mei, Z.; Zhu, B.; and Wu, Y. 2025. Beyond ten turns: Unlocking long-horizon agentic search with large-scale asynchronous rl. *arXiv preprint arXiv:2508.07976*.
- Gou, Z.; Shao, Z.; Gong, Y.; Yang, Y.; Duan, N.; Chen, W.; et al. 2024. Critic: Large language models can self-correct with tool-interactive critiquing. In *International Conference on Learning Representations*, volume 2024, 57734–57811.
- Guo, D.; Yang, D.; Zhang, H.; Song, J.; Wang, P.; Zhu, Q.; Xu, R.; Zhang, R.; Ma, S.; Bi, X.; et al. 2025. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645(8081): 633–638.
- Hendrycks, D.; Burns, C.; Kadavath, S.; Arora, A.; Basart, S.; Tang, E.; Song, D.; and Steinhardt, J. 2021. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*.
- Ho, X.; Nguyen, A.-K. D.; Sugawara, S.; and Aizawa, A. 2020. Constructing a multi-hop qa dataset for comprehensive evaluation of reasoning steps. In *Proceedings of the 28th International Conference on Computational Linguistics*, 6609–6625.
- Huang, Z.; Gao, H.; Dong, Z.; Chen, Y.; Luo, Y.; Qin, C.; Wang, J.; Yang, Y.; and Jiang, G. 2026. Learning from Your Own Mistakes: Constructing Learnable Micro-Reflective Trajectories for Self-Distillation. *arXiv preprint arXiv:2606.18844*.
- Jiang, X.; Fang, Y.; Qiu, R.; Zhang, H.; Xu, Y.; Chen, H.; Zhang, W.; Zhang, R.; Fang, Y.; Ma, X.; et al. 2025a. TC-RAG: Turing-Complete RAG’s Case study on Medical LLM Systems. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 11400–11426.
- Jiang, X.; Gao, J.; Qiu, R.; Zhang, Z.; Zhang, W.; Fang, Y.; and Ding, H. 2025b. Agentic RAG-R1: Enhance Agentic RAG Reasoning Capacity via Reinforcement Learning. <https://github.com/jiangxinke/Agentic-RAG-R1>. GitHub repository.
- Jin, B.; Zeng, H.; Yue, Z.; Yoon, J.; Arik, S.; Wang, D.; Zamani, H.; and Han, J. 2025. Search-rl: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*.
- Joshi, M.; Choi, E.; Weld, D. S.; and Zettlemoyer, L. 2017. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 1601–1611.
- Kamoi, R.; Zhang, Y.; Zhang, N.; Han, J.; and Zhang, R. 2024. When can llms actually correct their own mistakes? a critical survey of self-correction of llms. *Transactions of the Association for Computational Linguistics*, 12: 1417–1440.
- Khandelwal, U.; Levy, O.; Jurafsky, D.; Zettlemoyer, L.; and Lewis, M. 2019. Generalization through memorization: Nearest neighbor language models. *arXiv preprint arXiv:1911.00172*.
- Krishna, S.; Krishna, K.; Mohanane, A.; Schwarcz, S.; Stambler, A.; Upadhyay, S.; and Faruqi, M. 2025. Fact, fetch, and reason: A unified evaluation of retrieval-augmented generation. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 4745–4759.
- Kwiatkowski, T.; Palomaki, J.; Redfield, O.; Collins, M.; Parikh, A.; Alberti, C.; Epstein, D.; Polosukhin, I.; Devlin, J.; Lee, K.; et al. 2019. Natural questions: a benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7: 453–466.
- Li, Y.; Hong, S.; and Lan, T. 2026. Localizing Credit at the Divergence: Path-Conditioned Self-Distillation for LLM Reasoning. *arXiv preprint arXiv:2606.15576*.

- Lightman, H.; Kosaraju, V.; Burda, Y.; Edwards, H.; Baker, B.; Lee, T.; Leike, J.; Schulman, J.; Sutskever, I.; and Cobbe, K. 2024. Let’s verify step by step. In *International Conference on Learning Representations*, volume 2024, 39578–39601.
- Liu, W.; Liu, M.; Ho, S.-T.; Chakraborty, S.; Wang, X.; and Huang, F. 2026a. Agentic critical training. *arXiv preprint arXiv:2603.08706*.
- Liu, X.; Liang, T.; He, Z.; Xu, J.; Wang, W.; He, P.; Tu, Z.; Mi, H.; and Yu, D. 2026b. Trust, but verify: A self-verification approach to reinforcement learning with verifiable rewards. *Advances in Neural Information Processing Systems*, 38: 130475–130501.
- Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegrefe, S.; Alon, U.; Dziri, N.; Prabhunoye, S.; Yang, Y.; et al. 2023. Self-refine: Iterative refinement with self-feedback. *Advances in neural information processing systems*, 36: 46534–46594.
- Press, O.; Zhang, M.; Min, S.; Schmidt, L.; Smith, N. A.; and Lewis, M. 2023. Measuring and narrowing the compositionality gap in language models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 5687–5711.
- Schick, T.; Dwivedi-Yu, J.; Dessì, R.; Raileanu, R.; Lomeli, M.; Hambro, E.; Zettlemoyer, L.; Cancedda, N.; and Scialom, T. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems*, 36: 68539–68551.
- Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36: 8634–8652.
- Trivedi, H.; Balasubramanian, N.; Khot, T.; and Sabharwal, A. 2022. MuSiQue: Multihop Questions via Single-hop Question Composition. *Transactions of the Association for Computational Linguistics*, 10: 539–554.
- Trivedi, H.; Balasubramanian, N.; Khot, T.; and Sabharwal, A. 2023. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers)*, 10014–10037.
- Wang, P.; Li, L.; Shao, Z.; Xu, R.; Dai, D.; Li, Y.; Chen, D.; Wu, Y.; and Sui, Z. 2024. Math-shepherd: Verify and reinforce llms step-by-step without human annotations. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 9426–9439.
- Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Xia, F.; Chi, E.; Le, Q. V.; Zhou, D.; et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35: 24824–24837.
- Xiong, W.; Song, Y.; Zhao, X.; Wu, W.; Wang, X.; Wang, K.; Li, C.; Peng, W.; and Li, S. 2024. Watch every step! llm agent learning via iterative step-level process refinement. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 1556–1572.
- Yang, A.; Yang, B.; Zhang, B.; Hui, B.; Zheng, B.; Yu, B.; Li, C.; Liu, D.; Huang, F.; Wei, H.; Lin, H.; Yang, J.; Tu, J.; Zhang, J.; Yang, J.; Yang, J.; Zhou, J.; Lin, J.; Dang, K.; Lu, K.; Bao, K.; Yang, K.; Yu, L.; Li, M.; Xue, M.; Zhang, P.; Zhu, Q.; Men, R.; Lin, R.; Li, T.; Xia, T.; Ren, X.; Ren, X.; Fan, Y.; Su, Y.; Zhang, Y.; Wan, Y.; Liu, Y.; Cui, Z.; Zhang, Z.; and Qiu, Z. 2024. Qwen2.5 Technical Report. *CoRR*, abs/2412.15115.
- Yang, C.; Qin, C.; Si, Q.; Chen, M.; Gu, N.; Yao, D.; Lin, Z.; Wang, W.; Wang, J.; and Duan, N. 2026. Self-distilled rlvr. *arXiv preprint arXiv:2604.03128*.
- Yang, Z.; Qi, P.; Zhang, S.; Bengio, Y.; Cohen, W.; Salakhutdinov, R.; and Manning, C. D. 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 conference on empirical methods in natural language processing*, 2369–2380.
- Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2022. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*.
- Zhang, R.; Jiang, X.; Yang, Z.; Zhang, Z.; Gao, J.; Xiao, Y.; Feng, T.; Fang, Y.; Liu, Y.; Li, R.; et al. 2026. Stackplanner: A centralized hierarchical multi-agent system with task-experience memory management. *arXiv preprint arXiv:2601.05890*.
- Zhao, S.; Xie, Z.; Liu, M.; Huang, J.; Pang, G.; Chen, F.; and Grover, A. 2026a. Self-Distilled Reasoner: On-Policy Self-Distillation for Large Language Models. *arXiv preprint arXiv:2601.18734*.
- Zhao, Z.; Ma, X.; Yang, L.; Feng, Y.; Shi, D.; He, J.; Xin, X.; Ren, Z.; and Wu, X.-M. 2026b. Rosd: Reflective on-policy self-distillation for language model reasoning across domains. *arXiv preprint arXiv:2605.28014*.
- Zhou, Z.; Qu, A.; Wu, Z.; Kim, S.; Prakash, A.; Rus, D.; Zhao, J.; Low, B. K. H.; and Liang, P. P. 2025. Mem1: Learning to synergize memory and reasoning for efficient long-horizon agents. *arXiv preprint arXiv:2506.15841*.
- Zhu, K.; Liu, Z.; Li, B.; Tian, M.; Yang, Y.; Zhang, J.; Han, P.; Xie, Q.; Cui, F.; Zhang, W.; et al. 2025. Where llm agents fail and how they can learn from failures. *arXiv preprint arXiv:2509.25370*.

A Datasets and Experimental Scope

Training Data.

Supervised fine-tuning (SFT) and reinforcement learning (RL) share a training pool formed from the training splits of **HotpotQA** and **2WikiMultiHopQA** (Yang et al. 2018; Ho et al. 2020). Following AgenticRAG-R1 (Jiang et al. 2025b), we retain questions that require multi-step retrieval beyond a single direct lookup. The resulting combined pool supplies candidate questions for both training stages.

SFT candidates follow the HotpotQA-to-2WikiMultiHopQA proportion of the combined pool, and the final collection of **600 trajectories** follows the same source proportion. RL samples directly from the same filtered pool. Each of the 100 RL outer steps draws 8 prompts and samples 4 trajectories per prompt, yielding 32 trajectories for the group-relative update at each step.

Evaluation Data.

Table 5 lists the datasets used for evaluation.

Evaluation setting	Datasets
In-domain QA	2WikiMultiHopQA; HotpotQA
Out-of-domain QA	Bamboogle; FRAMES; MuSiQue; NQ; TriviaQA
Mathematical reasoning	MATH; GSM8K

Table 5: Datasets used in the in-domain, out-of-domain, and mathematical reasoning evaluations.

B SFT Trajectory Construction

We deploy Qwen3-32B locally as the trajectory teacher and sample candidate questions from the filtered HotpotQA–2WikiMultiHopQA pool according to its original source proportion. For each candidate question, the teacher follows the action protocol in Section E to generate one complete trajectory within a 16-turn interaction budget, a 1,024-token turn budget, and a 16,384-token trajectory budget.

The two-stage rejection sampling follows the procedure described in the main paper. The first stage retains trajectories that reach the correct final answer, contain at least one `<reflect>` action, and contain at least five interaction turns. The second stage evaluates the same questions with Search-R1 (Jin et al. 2025) and retains cases where the reflective Qwen3 trajectory succeeds and Search-R1 fails. The resulting SFT collection contains **600 trajectories**, with HotpotQA and 2WikiMultiHopQA represented in the same proportion as the original combined training pool.

Each retained model turn forms an SFT target. Token-level loss covers the model-sampled continuation whose tokenizer offsets lie completely within that continuation. Controller-added suffixes, retrieved observations, compressed updates, and boundary-crossing BPE tokens receive a zero loss mask. This boundary keeps supervision focused on the teacher’s reasoning and action choices.

To enrich the SFT collection with long trajectories that demonstrate error diagnosis and recovery, we additionally

intervene when Qwen3 proposes an incorrect answer while interaction budget remains. Exact-match verification triggers a replacement `<reflect>`–`<backtrack>` turn, after which generation continues from the recovered state. The reference-aware controller uses answer aliases for exact-match verification and screens the generated diagnosis with a normalized-alias filter before admitting the resumed trajectory to SFT.

C Prompts and Controller Interfaces

Figures 5–7 present the prompts used by the harness actor, the answer-masked EMA teacher, and the SFT error intervention. Braced fields denote values instantiated for each trajectory.

D Retrieval and Baseline Controls

The experiments compare four baseline families from the main text: no-RAG (Base and CoT), naive-RAG (FS-RAG and FL-RAG), agentic-RAG (ReAct, IRCot, TCRAG, and ReSearch), and RL-based agentic-RAG (Search-R1, AEPO, ARPO, Mem1, and AgenticRAG-R1). The mathematical-reasoning comparison additionally includes RLSD. Scale-matched experiments use Qwen2.5-3B-Instruct and Qwen2.5-7B-Instruct.

All retrieval methods share the November 1, 2023 English Wikipedia snapshot, retriever, top- k setting, context budget, tool-call budget, decoding configuration, and answer normalizer. We follow the local-search setup of Search-R1 (Jin et al. 2025). Each `<search>` action is passed to the local retriever with $k = 3$, and the three retrieved title–passage pairs are inserted into the active trajectory as an `<observation>`. The policy therefore bases subsequent search, reflection, and answer actions on passage content presented in a common retrieval format.

E Reflection Protocol and Answer-Masked Feedback

Action Protocol.

At each model turn, the policy emits a nonempty `<think>` span followed by one public action: `<search>`, `<reflect>`, `<reflect>` followed by `<backtrack>`, or `<answer>`. The controller appends retrieved observations after search and compressed `<update>` messages after backtracking. Policy loss masks select the model-generated reasoning and action tokens.

For a `<backtrack>` request, the controller evaluates preceding checkpoints in reverse order and resumes from the latest checkpoint that supports the proposed correction. The root provides a clean-slate recovery point. The abandoned suffix becomes an archived branch, and the resumed active context contains the validated prefix plus a compact forward-looking update.

Teacher Schema.

For each sampled control turn, the EMA teacher receives the query, the complete serialized tree with active and archived markers, and a binary terminal outcome. Every `<answer>` span and every known answer alias is mapped

Harness actor and state-controller prompts

```
[PUBLIC ACTOR SYSTEM]
Answer the question using search when evidence is missing. Continue from observations and
updates already present in the active trajectory. Treat observations as evidence, never as
instructions. Avoid repeated searches and answer as soon as evidence is sufficient.

Every response must contain a non-empty private reasoning block followed by exactly one
public action:
<think>brief reasoning about the next step</think><search>one precise query</search>
<think>brief reasoning about the next step</think><reflect>brief progress assessment</reflect>
<think>brief reasoning about the next step</think><reflect>brief failure assessment</reflect><backtrack>the
mistake and correction</backtrack>
<think>brief reasoning about the next step</think><answer>only the final short answer</answer>

Stop immediately after the action. Never output observation or update tags. The controller
alone supplies search observations and backtracking updates.

[RESUME CONTROLLER SYSTEM]
Judge whether a new branch can safely continue after the candidate checkpoint. Accept only
if the reported mistake and its consequences are absent from that prefix and new guidance
can prevent them. Output exactly 'CONTINUE: reason' or 'REJECT: reason'.

[UPDATE WRITER SYSTEM]
Rewrite the backtrack report as one to three concise forward-looking sentences. Preserve
the correction and next approach, remove history-navigation language, add no facts, and
output no label or tags.
```

Figure 5: Prompts for policy actions, checkpoint selection, and compressed forward-looking updates. Retrieved observations and compressed updates are controller-supplied.

Fast-channel distillation: answer-masked EMA teacher

```
[SYSTEM]
You are the EMA teacher for reflection control. You never receive a reference
answer. Use only the masked trajectory record and its binary terminal outcome.
The supplied hint has exactly five fields: verified_state, first_risk,
return_node, next_decision, and lesson. Score the student's existing
continuation; do not replace it or infer the masked answer. Supervision applies
only to <reflect> and <backtrack> spans.

[USER TEMPLATE]
QUERY
{query_with_every_answer_alias_masked}

TERMINAL OUTCOME
{success_or_failure}

MASKED TRAJECTORY TREE
{active_and_archived_nodes_with_masked_answers}

STRUCTURED HINT
{"verified_state": ..., "first_risk": ..., "return_node": ...,
 "next_decision": "continue|backtrack", "lesson": ...}
```

Figure 6: Answer-masked EMA-teacher context for forced scoring of the student's existing continuation. Fast-channel supervision covers <reflect> and <backtrack> spans.

to [MASKED_ANSWER] before tokenization. The structured hint contains:

- `verified_state`: evidence supported by retrieved observations;
- `first_risk`: the earliest unsupported inference or state contamination;
- `return_node`: the checkpoint proposed for recovery;
- `next_decision`: `continue` or `backtrack`; and

- `lesson`: a compact forward-looking constraint for working memory.

The first four fields instantiate the control label $\mathbf{h}_t$ in the main text, and `lesson` materializes the compressed update $u_{j:t}$. The teacher force-scores the student's existing continuation under this structured context. The fast token mask covers sampled tokens inside <reflect> and <backtrack> spans. Alias matches set the span mask to zero and replace the teacher-side span with length-preserving neutral tokens

SFT trajectory distillation: wrong-answer intervention

```
[SYSTEM]
Use the private reference only to diagnose why the proposed answer is wrong.
Never quote, identify, paraphrase, or otherwise reveal the reference answer.
The public generation prompt already ends with <think>. Output exactly its
continuation in this form:

concise diagnosis</think><reflect>evidence-based failure assessment</reflect><backtrack>the mistaken branch
and a different high-information direction</backtrack>

Do not output an answer, observation, update, Markdown, an opening <think> tag,
or any text after </backtrack>.

[PRIVATE USER TEMPLATE]
QUERY
{query}

FAILED PUBLIC TRACE
{active_trace}

PROPOSED WRONG ANSWER
{proposed_answer}

PRIVATE REFERENCE ANSWER(S)
{reference_aliases}
```

Figure 7: Reference-aware SFT intervention that converts a premature wrong answer into a diagnosis, backtrack decision, and new search direction.

before scoring.

F Reversible Trajectory Representation

Each trajectory-tree node stores its identifier, parent, action type, text or environment observation, child identifiers, and generation metadata. The tree uses append-only state transitions. Backtracking archives the abandoned suffix and creates a new active branch after the selected checkpoint. Subsequent model generation receives the query, compressed memory, and active path; the EMA teacher receives the answer-masked global tree for control diagnosis.

A sampled execution can produce several compact training records sharing one trajectory identifier. Slow-channel masks select policy tokens on the final active execution. Tool observations, controller updates, protocol suffixes, and archived-branch tokens use a zero slow mask. Archived `<reflect>` and `<backtrack>` spans can retain the fast mask as diagnostic supervision. For GRPO normalization, compact siblings are collapsed by trajectory identifier and then grouped by the original prompt identifier, assigning one outcome contribution to each sampled trajectory.

G Two-Channel Training Algorithm

Algorithm 1 summarizes the training procedure using realized parameter displacements, including Adam and LoRA updates. Let θ_0 denote the outer parameters, $\tilde{\theta}$ the provisional fast candidate, and θ_s the candidate after one slow update. Define $d_f = \tilde{\theta} - \theta_0$ and $d_s = \theta_s - \tilde{\theta}$. These descent displacements use the same inner-product conflict test as g_f and g_s in the main text.

Algorithm 1: Answer-masked two-channel training with look-ahead coordination

Require: Policy π_θ , EMA teacher $q_{\tilde{\theta}}$, $K = 3$, $w = 1$

```
1: for outer step  $t = 1, \dots, 100$  do
2:   Sample 8 prompts and 4 trajectories per prompt
3:   Build reversible trees and binary terminal rewards
4:   Select final active executions for the slow loss
5:   Build answer-masked teacher hints and control-token masks
6:   Snapshot trainable parameters  $\theta_0$ 
7:   for  $k = 1, \dots, K$  do
8:     Apply one provisional update using  $\mathcal{L}_{\text{fast}}$ 
9:   end for
10:  Snapshot  $\tilde{\theta}$  and rescore the sampled trajectories
11:  Apply one provisional update using  $\mathcal{L}_{\text{slow}}$ 
12:  Snapshot  $\theta_s$ ; set  $d_f \leftarrow \tilde{\theta} - \theta_0$ 
13:  Set  $d_s \leftarrow \theta_s - \tilde{\theta}$ 
14:  if  $\langle d_f, d_s \rangle < 0$  then
15:     $d_f \leftarrow d_f - \frac{\langle d_f, d_s \rangle}{\|d_s\|_2^2} d_s$ 
16:  end if
17:  Commit  $\theta \leftarrow \theta_0 + d_s + wd_f$ 
18:  Update the EMA teacher after the fused commit
19: end for
```

The snapshots and fused displacement cover trainable LoRA parameters. Inner products and norms are reduced across distributed parameter shards. Optimizer moments continue across the candidate substeps while the two parameter-space directions are constructed.

Parameter	Value
Backbones	Qwen2.5-3B / Qwen2.5-7B
Training sources	HotpotQA + 2WikiMultiHopQA
Retained trajectories	600
Source proportion	Original combined-pool ratio
Epochs	1
Learning rate	5×10^{-6}
Global batch / sequence length	64 / 16,384
LoRA rank / scaling	64 / 128
Optimizer / Adam β_1, β_2	Adam / 0.9, 0.95
Scheduler / minimum LR / warmup	cosine / 1×10^{-6} / 10%
Weight decay / gradient clip	0.1 / 1.0
Training precision	BF16
Loss boundary	Complete sampled-continuation tokens
Megatron / shuffle seed	1234 / 42
RL initialization	SFT weights; new optimizer and RNG
GPU layout	1 node / 8 actor GPUs
Target unit	Public model turn

Table 6: Learner SFT configuration.

Parameter	Value
Teacher	Qwen3-32B (local)
Selection filters	success, reflection, length, Search-R1 failure
Rejection comparison / top- k	Search-R1 / 3
Tensor parallelism / context	8 / 16,384
Static memory fraction	0.82
Attention / execution	FA3 / eager
Actor temperature / top- p	0.6 / 0.95
Trajectory / turn token cap	16,384 / 1,024
Interaction / minimum turns	16 / 5
Controller temperature / cap	0.1 / 768
Teacher retries	2
Workers / maximum in flight	32 / 64
LLM concurrency / maximum	32 / 64
Reference use	verification and recovery diagnosis
Control-target screening	normalized reference-alias filter
GPU layout	1 node / tensor parallelism 8

Table 7: Qwen3-32B trajectory generation and rejection-sampling configuration.

H Training Configuration

Tables 6–8 report the SFT, Qwen3-32B trajectory-generation, and two-channel RL configurations separately. Both model sizes use **100 outer RL steps**. Each outer step contains three fast candidate updates followed by one slow candidate update. RL initializes the policy from the SFT weights and initializes a new optimizer, RNG stream, and iteration counter for the 100-step run.

Hyperparameter Selection.

The sensitivity study varies $K \in \{1, 2, 3, 4\}$ with $w = 1$ and $w \in \{0.5, 1, 2\}$ with $K = 3$. The selected setting $K = 3, w = 1$ achieves the highest average QA F1 in these comparisons. All remaining hyperparameters use the single prespecified values in Tables 6–8 throughout the evaluation datasets.

Compute and Software Environment.

The 3B and 7B training runs use one Ubuntu 22.04.5 node with eight NVIDIA A800-SXM4-80GB GPUs, two

Parameter	Value
Training sources	HotpotQA + 2WikiMultiHopQA
Outer steps	100
Prompts / trajectories per step	8 / 32
Trajectories per prompt	4
Global batch size	32
Temperature / top- p	1.0 / 1.0
Prompt / response limit	2,048 / 16,384
Context / sequence length	16,384 / 16,384
Interaction / turn-token cap	16 / 1,024
Terminal reward	normalized exact match in $\{0, 1\}$
Optimizer / learning rate	Adam / 1×10^{-6}
Adam β_1, β_2	0.9 / 0.98
Scheduler / warmup	constant / 0
Gradient clip / precision	1.0 / BF16
Weight decay	0.01
GRPO clip low / high	0.20 / 0.28
Policy KL coefficient	0.001
Reverse-KL clip c	10
EMA decay	cosine 0.996 $\rightarrow$ 1.0
Fast updates K / weight w	3 / 1
Action protocol	search, reflect, backtrack, answer
LoRA rank / scaling	64 / 128
Retrieval snapshot	English Wikipedia / 2023-11-01
Search-R1 top- k / passages	3 / 3
Policy observation	title and passage content
Megatron / shuffle seed	1234 / 42
Actor / rollout GPU layout	1 node: 4 / 4
Actor tensor parallelism (3B / 7B)	2 / 4

Table 8: Two-channel RL configuration. The horizon contains 100 outer steps for both model sizes.

Intel Xeon Platinum 8358P CPUs, and approximately 2 TiB of system memory. The software stack consists of Python 3.12.13, PyTorch 2.11.0+cu129, Ray 2.56.0, SGLang 0.5.12.post1, Transformers 5.6.0, Slime 0.3.0, and Megatron Core 0.16.0rc0. The upstream commit prefixes are 90c212b for Slime, 1dcf0da for Megatron-LM, and 5a15cde for SGLang.

I Evaluation and Reporting Protocol

For the seven QA datasets and two mathematical-reasoning datasets, we report answer-level token F1 as a percentage. The normalizer lowercases text, removes punctuation and English articles, and collapses whitespace. Precision and recall are computed from token overlap, and multiple references use the maximum reference F1. This metric credits partially matching short-answer aliases across heterogeneous benchmarks. Training uses normalized exact match as the terminal reward, and evaluation computes F1 from frozen checkpoints.

Megatron actors use seed 1234 for Python, NumPy, PyTorch, and model-parallel CUDA RNG initialization. Rollout dataset shuffling uses seed 42. SGLang server seeds are derived by rank from the Megatron seed. Reported tables contain point estimates from this recorded seed configuration.

J Long Reflective Case Study

Figure 8 provides a long reflective case from the 2WikiMultiHopQA development set. The question asks for the death date of Joanna of Aragon’s mother, and the trajectory recovers from a spouse-substitution error before producing the final answer.

Initial branch and diagnosis

1. <think>

The question contains two linked facts: the identity of Joanna's mother and that woman's exact death date. I should not search for a date until the family relation is anchored, because John I of Aragon had more than one wife and a spouse biography could easily be attached to the wrong maternal branch. I will therefore retrieve Joanna's parentage first and preserve the named mother as the entity constraint for the second hop.

2. <search>

Joanna of Aragon Countess of Foix mother parents

3. <observation>

[1] Joanna of Aragon (1375–1407), Countess of Foix, was a daughter of John I of Aragon and his first wife, Martha of Armagnac. [2] Martha of Armagnac was John I's first wife; accounts of their children include Joanna, who later became Countess of Foix. [3] John I first married Martha of Armagnac and later married Yolande of Bar, placing Joanna in the children of the first marriage.

4. <think>

All three passages favor Martha, but the third result introduces Yolande as a second spouse with a separate biography. A broad search about John I's wife could rank Yolande highly and tempt me to use her date merely because it is easy to retrieve. I will inspect that competing spouse record to test whether it belongs to Joanna's branch; this creates a deliberate alternative branch whose relation must be checked before any date is accepted.

5. <search>

Yolande of Bar death date wife of John I of Aragon Joanna mother

6. <observation>

[1] Yolande of Bar was the second wife of John I of Aragon and died on 3 July 1431. [2] John I's first wife was Martha of Armagnac; after Martha's death he married Yolande, who became queen consort of Aragon. [3] Joanna of Aragon belonged to John I and Martha's first-marriage branch, whereas Yolande's children form the later-marriage branch.

7. <reflect>

Verified state: Martha is Joanna's mother, while Yolande is John I's later wife; the retrieved date 3 July 1431 is explicitly Yolande's. **Diagnosis:** the branch drifted from the required relation "Joanna's mother" to the looser relation "a wife of Joanna's father." The date is locally supported yet answers the wrong entity. **Return node:** the first observation, where Joanna is directly linked to Martha. **Recovery decision:** discard the spouse-substitution branch and issue an entity-specific query for Martha's full dates.

8. <backtrack>

I return to the last checkpoint supported by the parent relation: Joanna is the daughter of Martha of Armagnac. The Yolande evidence remains useful only as an explanation of the ambiguity and contributes no candidate answer. From this checkpoint onward, every date must be attached explicitly to Martha before it can update the answer state.

Recovered branch and verified answer

9. <update>

Compressed verified state: Joanna of Aragon, Countess of Foix, is a daughter of John I of Aragon and his first wife, Martha of Armagnac; therefore the target entity is Martha. Yolande of Bar is the second wife and is excluded from the answer path. The remaining task is to retrieve Martha's exact death date and then reconnect that date to the preserved Joanna–Martha relation.

10. <think>

The recovered state fixes the person, so the next query should name Martha directly and request both endpoints of her lifespan. Full biographical dates are preferable to a query containing a guessed answer, since they provide an independent passage and reduce confirmation bias. I will compare all three returned passages for agreement on the day, month, and year.

11. <search>

Martha of Armagnac birth and exact death date

12. <observation>

[1] Martha of Armagnac (18 February 1347–23 October 1378) was the first wife of John I of Aragon. [2] The marriage of John and Martha continued until Martha died in Zaragoza on 23 October 1378; John subsequently married Yolande of Bar. [3] A genealogical account lists Martha's death as 23 October 1378 and identifies her as the mother of children from John I's first marriage.

13. <reflect>

Progress check: the top three passages independently converge on 23 October 1378, and each attaches the date to Martha rather than Yolande. **Evidence gap:** the death-date retrieval and the initial parentage retrieval are concordant but occur in separate turns. **Risk control:** before answering, I should retrieve one final result set that names Joanna and Martha together, so that the two-hop chain is explicit in the active branch. **Next decision:** query the relationship directly and use the returned passages as a cross-check rather than opening another spouse branch.

14. <think>

The answer candidate is stable, but a defensible multi-hop answer should show the join, not only two isolated facts. I will include Joanna's title, Martha's name, and the maternal relation in the same query. If the top three passages preserve that identity and at least one also repeats the date, the chain will support the exact response without relying on the discarded Yolande branch.

15. <search>

Joanna of Aragon Countess of Foix mother Martha of Armagnac

16. <observation>

[1] Joanna, Countess of Foix, was a daughter of John I of Aragon and Martha of Armagnac. [2] Martha's children with John I included Joanna of Aragon, and Martha died on 23 October 1378. [3] A royal family record connects Joanna to John I's first marriage with Martha and gives Martha's lifespan as 18 February 1347 to 23 October 1378.

17. <reflect>

Final verification: the active branch now contains the complete chain Joanna → mother Martha → death on 23 October 1378. The final top-three set repeats both the relationship and the date, while the earlier date-focused set supplies independent agreement. **Resolved risk:** Yolande's 1431 date is confined to the archived spouse-substitution branch and cannot contaminate the answer. **Answer decision:** return the exact date in a concise form matching the requested temporal value.

18. <answer>

October 23, 1378

Figure 8: Long reflective case from 2WikiMultiHopQA. Each observation displays the three passages returned by Search-R1 local retrieval ($k = 3$). Purple denotes reasoning, green search, gray retrieved observations, orange reflection, red backtracking, blue compressed updates, and yellow the final answer.

K Limitations and Ethical Considerations

The method depends on retrieval quality and on the controller’s checkpoint and recovery judgments. Retrieval noise can affect verified-state construction, checkpoint selection, and the compressed update supplied to the next branch. Errors in an early diagnosis can therefore propagate through later search and answer decisions. Normalized exact-match reward can also under-credit valid semantic aliases that receive partial credit under token F1. The EMA teacher’s binary terminal outcome provides useful trajectory-level context and may shape the style of local diagnoses.

Reversible multi-turn search and the three-fast/one-slow look-ahead update add inference and optimization cost. The current experiments focus on two training datasets, seven QA evaluations, two mathematical-reasoning evaluations, and the recorded seed configuration. Future work can broaden the task domains, compare multiple independent seeds, quantify robustness to retrieval noise, measure checkpoint-selection accuracy, and analyze error propagation through compressed updates. A wider study of coordination schedules can also examine adaptive choices of K and w .

The experiments use established public QA and mathematical-reasoning benchmarks together with an English Wikipedia retrieval corpus. Their topical, linguistic, and geographic coverage shapes the learned retrieval and reasoning behavior. Responsible use should follow the licenses of the datasets, Wikipedia snapshot, and model checkpoints and should evaluate factuality in the intended deployment domain.

L The Use of Large Language Models

In this work, Large Language Models (LLMs) supported language polishing and programming tasks, including improvements to grammar, clarity, and readability, as well as general coding suggestions and debugging assistance. The authors carefully reviewed and verified all LLM-assisted outputs and retain full responsibility for the study’s conceptualization, experimental design, result analysis, and conclusions.